Microsoft AI Unlocked | Idea Submission

Architectural Intelligence Ledger

AIL

Your Codebase Has a Story, Now You Can Read It.

Institute	Manipal Institute of Technology
Challenge Area	Track 1: The Productivity Flow
Team Name	DSP
Team Lead Email	rudra3.mitmpl2023@learner.manipal.edu

Project Title

Architectural Intelligence Ledger (AIL)

AI-powered codebase understanding and architectural knowledge management, built into your editor.

The Team

We are a **software engineering team from Manipal Institute of Technology** with backgrounds in **systems design, applied AI, and full-stack development**. Through working on complex technical projects together, we repeatedly encountered the same productivity bottleneck: understanding **why existing systems were built the way they were**. That firsthand experience motivated us to build a tool that makes codebase intelligence instantly accessible to every developer on a team.

The Concept

According to a **McKinsey & Company study (2024)**, developers spend up to **40% of their time** just understanding existing codebases. When engineers leave, the **architectural reasoning** behind critical decisions leaves with them. Git shows **what** changed but never **why**. Documentation goes stale. New hires spend weeks building a mental model before they can contribute. This is one of the **largest measurable drains on engineering productivity** in modern software organizations.

AIL is a **VSCode extension** that transforms any GitHub repository into a **living, queryable knowledge graph**. It ingests the codebase, performs deep static analysis via **AST parsing**, and synchronizes that structural map with the full **Git commit and PR history**. Developers interact with this graph through a **natural-language agent** directly inside VSCode, getting factual, relationship-aware answers grounded in the actual code and its history.

Unlike AI coding assistants that use **retrieval-augmented generation** at the file level, AIL reasons over the **entire system graph**. It understands that a change in one module propagates to services three hops away. It knows **why** an architectural decision was made because it retrieved that context from the **actual PR** where the decision was discussed. It does not guess; it **traverses**.

Target Audience or Market

AIL is built for anyone who reads and works with code. **Students and independent developers** use it to understand open-source projects and personal codebases faster. **Engineering teams at mid-to-large companies** use it where codebases exceed 100k lines, team rotations are frequent, and onboarding cost is measurable. **Fast-scaling startups** use it where hypergrowth outpaces documentation, and **regulated industries** (FinTech, HealthTech) use it for audit trails and architectural change records.

Segment	Why They Need AIL
Students and Personal Devs	Understand any open-source or personal codebase instantly without reading every file.
Enterprise Engineering	Architectural governance, drift detection, automated compliance documentation.
Fast-Scaling Startups	Onboarding acceleration and knowledge preservation during rapid headcount growth.
FinTech / HealthTech	Audit trails for system changes and automated ADRs before regulated deployments.
IT Consulting Firms	Rapid comprehension and documentation of inherited legacy codebases for clients.

The Global DevOps and Developer Experience market was valued at $10.4 billion in 2023 and is projected to reach **$25.5 billion by 2028** (MarketsandMarkets, 2023). The adjacent **Knowledge Management Software market** adds a further $14.4 billion globally (Grand View Research, 2024). AIL sits precisely at the intersection of these two markets.

Personas

Arjun Mehta, Senior Tech Lead, 34

Attribute	Detail
Core Problem	Spends 6-8 hours per week answering 'how does this work?' questions. Context-switching destroys strategic output.
How AIL Helps	AIL becomes his automated knowledge bridge. Every structural change is documented in real-time and any teammate can query it directly from VSCode, without interrupting Arjun.

Priya Shankar, Junior Software Engineer, 24

Attribute	Detail
Core Problem	Joined a 400+ microservice platform. No one has time to explain the system; the README points to a 2021 diagram.
How AIL Helps	She types 'Explain the order fulfilment flow to me like I am new' inside VSCode. AIL returns a narrative walkthrough grounded in actual codebase history. She files her first meaningful PR by Day 5.

How It Works

AIL is delivered as a **VSCode extension**. On activation, it connects to the opened repository, runs the analysis pipeline via **Azure-hosted microservices**, and surfaces an interactive **Knowledge Graph panel** and a **conversational agent sidebar** directly within the editor. Intelligence lives where developers already work.

Layer 1 | Repository Ingestion and Language Detection

- **Workspace Connect:** Accepts the active VSCode workspace or a GitHub URL. Auto-detects language, stack, and entry points.
- **Execution Model:** Identifies project type (web service, CLI, library, batch job) and parses dependency manifests.
- **Baseline Metrics:** Establishes structural baseline: file count, LOC, language distribution, and dependency depth.

Layer 2 | AST-Based Static Analysis Engine

- **Entity Extraction:** Parses all source files with Tree-sitter, extracting files, classes, functions, API endpoints, database models, imports, and exception handlers.
- **Relationship Graph:** Builds a typed graph: call edges, import edges, inheritance edges, and data-flow edges across shared state and databases.
- **Control Flow:** Traces execution from entry points, mapping the happy path, conditional branches, recursive patterns, and circular dependencies.
- **Code Metrics:** Computes cyclomatic complexity, dependency depth, and code churn velocity from Git history.

Layer 3 | Temporal Git Intelligence

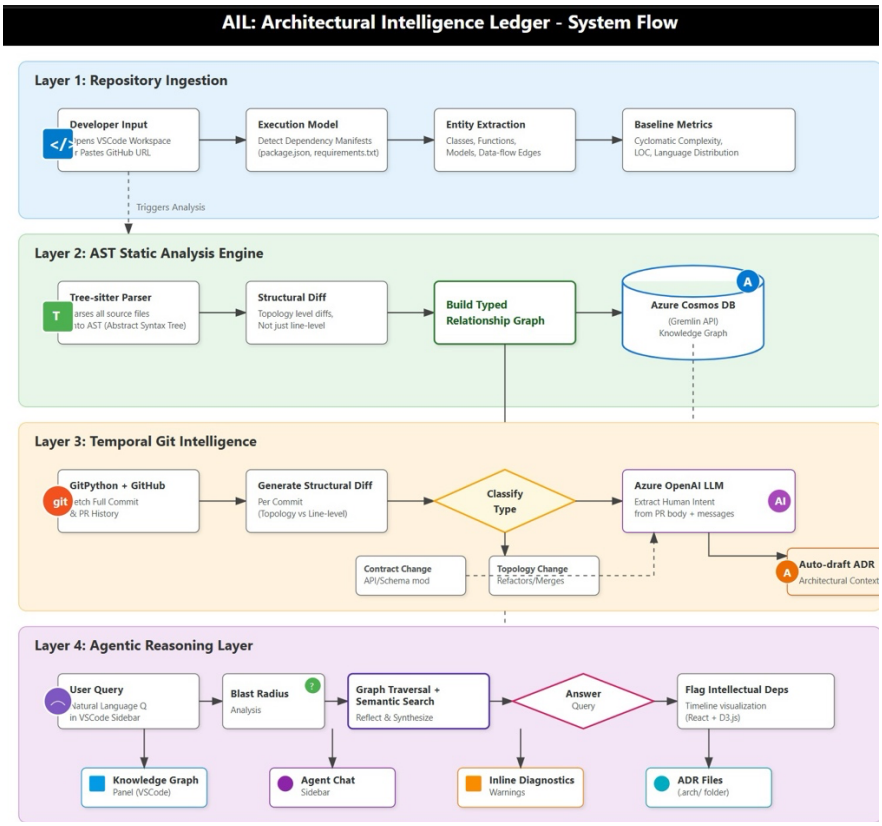
- **Commit Parsing:** Fetches the full Git history and parses every commit: message, author, timestamp, and changed file set.
- **Structural Diff:** Generates a topology-level diff per commit, tracking service merges, API contract changes, and communication pattern shifts. This is the core architectural evolution record of the repository.
- **Change Classification:** Classifies changes as: Boundary Changes (how services communicate), Contract Changes (API or schema mutations), and Topology Reorganizations (structural restructuring such as monolith splits).
- **Archaeological Context:** Uses an LLM to parse PR bodies and commit messages, extracting human intent behind structural changes and storing it linked to affected graph nodes. Preserves the 'why' alongside the 'what'.
- **ADR Generation:** Auto-drafts Architectural Decision Records when Boundary Changes or Topology Reorganizations are detected, committed to a .arch/ folder.
- **Expertise Map:** Tracks Git blame data to identify which engineers have deepest authorship over each architectural module.

Layer 4 | Agentic Reasoning Layer (VSCode Sidebar)

The agent operates as a **multi-step reasoning system**: it plans the query, traverses the knowledge graph, reflects on completeness, and synthesizes a grounded answer. It does **not retrieve snippets; it traverses relationships**. Four interaction modes:

- **Onboarding Chat:** Answers natural-language questions using graph traversal, not keyword search. Answers are cited to specific nodes, files, and historical commits. ('Where is authentication handled?', 'Explain the data flow from API to database')
- **Blast Radius Analysis:** Reverse-traverses the call graph to enumerate every function, endpoint, and downstream service affected by a proposed change, producing a prioritized impact report.

- **Architectural Evolution Queries:** Answers how the entire architecture has evolved over time ('When did we move from microservices to a monolith?'). Surfaces structural diff timeline, commit intent, and ADR history.
- **Architectural Health Monitor:** Runs on every save or commit. Flags circular dependencies, anti-patterns, and architectural drift as inline VSCode diagnostics.



Core Technologies

Backend and Analysis

Technology	Role
Python 3.11 + FastAPI	Core analysis microservices: ingestion, AST processing, Git parsing, ADR generation.
Tree-sitter	Incremental AST parser supporting 15+ languages. Backbone of entity and relationship extraction.
GitPython + GitHub API	Git history ingestion, commit parsing, blame traversal, and PR body retrieval.
Celery + Redis	Async task queue for long-running analysis jobs, keeping the extension responsive.

Graph Database

Technology	Role
Azure Cosmos DB (Gremlin API)	Primary Knowledge Graph store. Gremlin traversal powers multi-hop queries like blast radius and call chain tracing.
Azure Cache for Redis	Caches hot graph subgraphs and embedding vectors to reduce repeated query latency.

AI and Agentic Layer

Technology	Role
Azure OpenAI Service	Core LLM for Archaeological Context extraction, ADR drafting, and natural-language query answering. No training on customer code.
Azure AI Foundry	Orchestrates the multi-step agent pipeline: plan, graph query, traverse, reflect, synthesize.
Semantic Embeddings	Dense vector embeddings for all code entities, enabling hybrid graph traversal with semantic similarity search.
LangChain	Agent tool definitions, prompt templating, and structured output parsing wrapping Gremlin queries.

VSCode Extension and Visualization

Technology	Role
VSCode Extension API	Primary delivery surface. Hosts the Knowledge Graph panel, agent sidebar, and inline diagnostics.
React 18 + TypeScript	Webview components for graph explorer, chat interface, and ADR viewer.
D3.js + React Flow	Force-directed graph layout and structured flow diagrams for call chains and control flow.

Infrastructure

Technology	Role
Azure Container Apps	Hosts all backend microservices in auto-scaling containers.
Azure API Management	Unified gateway handling authentication, rate limiting, and routing.
Azure Monitor + App Insights	Distributed tracing, query performance monitoring, and alerting.
Azure Key Vault	Secure storage for API credentials and database connection strings.

The Business Plan

Revenue Model

Tier	Model
Free	1 repo, public only. Graph visualization and basic chat. Targets students and OSS maintainers.
Pro (Rs. 499/mo)	Up to 10 private repos. Full agent, ADR generation, blast radius analysis, and onboarding narratives.
Team (Rs. 2,999/mo per 5 seats)	Unlimited repos, expertise mapping, architectural health digest, and CI/CD integration.
Enterprise (Custom)	Self-hosted on Azure, SSO, data residency guarantees, and dedicated support.

Go-to-Market Strategy

- **Developer-Led Growth:** Launch the free VSCode extension on the VSCode Marketplace and Product Hunt. Target individual engineers, students, and OSS maintainers.
- **Bottoms-Up Enterprise:** Free and Pro users inside companies generate internal adoption. Three or more users from the same org on Pro triggers a Team tier conversion motion.
- **Ecosystem Integration:** Build a companion GitHub Action that posts AIL architectural health summaries directly on PRs, exposing the product to entire engineering teams with zero setup.

Competitive Differentiation

Capability	Cursor / Copilot / Claude	AIL
Scope	File or function level (local)	Full system topology (global)
History	Sees code now only (ephemeral)	Sees code across all commits (temporal)
Search	RAG: snippet retrieval	Graph traversal: relationship-aware path queries
The Why	Infers from current code (hallucination risk)	Retrieved from actual PR and commit history (factual)
Output	Code completion	Diagrams, ADRs, impact reports, narratives
Onboarding	Not a primary use case	Core product value proposition

Additional Information

MVP Scope (Hackathon Prototype)

- VSCode extension prototype that ingests a public Python GitHub repository from the active workspace.
- Tree-sitter AST analysis producing a live, interactive D3.js Knowledge Graph rendered in the VSCode panel.
- Git history parsing with LLM-powered Archaeological Context extraction, linking commit intent to specific graph nodes.
- Working chat agent in the VSCode sidebar answering: 'Explain this repo to me', 'Where is authentication handled?', 'What breaks if I change this function?', and 'How has the architecture of this codebase evolved over time?' using graph-traversal-backed responses grounded in both current code and historical structural diffs.
- Automatic ADR generation for at least one detected Boundary Change or Topology Reorganization in the repository history.